

DOMAIN 1 — DSA + Coding + Problem Solving

1. Problem Comprehension & Algorithmic Thinking

What Interviewers Expect

Candidates should:

- Clarify ambiguous requirements
 - Ask intelligent questions
 - Identify constraints
 - Think about edge cases before coding
 - Choose appropriate data structures
 - Explain reasoning before implementation
-

How To Approach Any Problem

Step 1 — Clarify Requirements

Questions to ask:

- What are the input constraints?
 - Can input be null or empty?
 - What is the expected output?
 - Are duplicates allowed?
 - Is ordering important?
 - What are performance expectations?
 - Is the system memory-sensitive?
-

Step 2 — Discuss Edge Cases

Examples:

- Empty input
- Null values
- Duplicate data

- Large datasets
 - Invalid requests
 - Integer overflow
 - Concurrent access
 - Cache misses
 - Circular dependencies
-

Step 3 — Explain High-Level Approach

Candidate should:

- Explain brute force first
 - Improve incrementally
 - Justify optimized solution
 - Explain why chosen DS/Algo fits
-

Important Data Structures To Master

Arrays

Topics:

- Sliding window
- Prefix sum
- Two pointer
- Kadane algorithm
- Matrix traversal

Strings

Topics:

- Frequency counting
- Pattern matching
- Palindrome
- Hashing

- Substring optimization

HashMap / HashSet

Topics:

- $O(1)$ lookup
- Frequency maps
- Deduplication
- Cache design

Stack

Topics:

- Balanced brackets
- Monotonic stack
- Expression evaluation
- Next greater element

Queue / Deque

Topics:

- BFS
- Sliding window max
- Rate limiting

Linked List

Topics:

- Reversal
- Cycle detection
- Merge operations
- LRU cache

Trees

Topics:

- Traversals
- BST operations

- Lowest common ancestor
- Height/depth problems

Graphs

Topics:

- BFS/DFS
- Topological sort
- Shortest path
- Cycle detection
- Connected components

Heap / Priority Queue

Topics:

- Top K problems
- Scheduling
- Median finder
- Merge K sorted lists

2. Complexity Analysis (Time & Space)

Interview Expectation

Candidates should:

- Explain time complexity clearly
- Explain space complexity
- Compare approaches
- Identify bottlenecks
- Suggest optimizations

Important Big-O Concepts

Complexity Meaning

$O(1)$	Constant time
$O(\log N)$	Binary search style
$O(N)$	Linear scan
$O(N \log N)$	Sorting
$O(N^2)$	Nested loops
$O(2^N)$	Exponential recursion

Common Optimization Patterns

Replace Nested Loop Using HashMap

Example:

- Two sum
- Duplicate detection

Sorting + Two Pointer

Example:

- Triplet problems
- Interval merge

Sliding Window

Example:

- Longest substring
- Fixed-size window problems

Memoization / DP

Example:

- Fibonacci
- Knapsack

- Grid traversal
-

What Strong Candidates Do

Strong candidates:

- Start with brute force
 - Improve progressively
 - Quantify trade-offs
 - Explain scalability impact
-

3. Code Implementation & Quality

Interview Expectations

Code should be:

- Readable
 - Modular
 - Maintainable
 - Production-ready
 - Cleanly structured
-

Production-Grade Coding Practices

Naming Standards

Good:

- `customerOrderMap`
- `activeSessions`
- `fetchInventory()`

Bad:

- `x`
- `temp`
- `data1`

Modularity

Break logic into:

- Validation layer
- Business logic
- Helper functions
- Utility methods

SOLID Principles

Candidates should understand:

S — Single Responsibility Principle

One class should have one responsibility.

O — Open Closed Principle

Open for extension, closed for modification.

L — Liskov Substitution Principle

Child should replace parent safely.

I — Interface Segregation Principle

Avoid fat interfaces.

D — Dependency Inversion Principle

Depend on abstractions.

Java Coding Best Practices

Exception Handling

- Use custom exceptions
- Avoid generic Exception
- Add meaningful logging

Immutability

- Prefer immutable objects

- Use final where possible

Collections

- Choose correct collection
- ArrayList vs LinkedList
- ConcurrentHashMap vs HashMap

Streams API

Know:

- filter
 - map
 - reduce
 - groupingBy
 - collectors
-

4. Exception Handling & Edge Cases

Interview Expectation

Candidates should:

- Think defensively
 - Validate inputs
 - Handle failures gracefully
 - Prevent crashes
-

Important Areas

Null Handling

- Optional usage
- Null checks
- Defensive coding

Concurrency Issues

- Race conditions

- Deadlocks
- Thread safety

Invalid Inputs

- Validation
- Error responses
- Retry policies

Distributed Failures

- Timeout
 - Partial failures
 - Retry storm
 - Idempotency
-

Advanced Concepts

Idempotency

Repeated requests should not create duplicate effects.

Used in:

- Payments
 - Order processing
 - Inventory updates
-

Retry Mechanism

Types:

- Fixed retry
 - Exponential backoff
 - Circuit breaker
-

5. Testing & Verification

What Interviewers Expect

Candidates should:

- Test manually
 - Verify edge cases
 - Validate assumptions
 - Explain testing strategy
-

Types Of Testing

Unit Testing

Focus:

- Business logic
- Utility methods
- Edge cases

Integration Testing

Focus:

- DB interaction
- API communication
- Kafka events

End-to-End Testing

Focus:

- Complete workflow validation
-

Important Testing Concepts

Mocking

Tools:

- Mockito
- WireMock

Assertions

Candidates should know:

- Positive cases
 - Negative cases
 - Boundary conditions
-

6. Communication & Collaboration

What Strong Candidates Demonstrate

- Structured thinking
 - Calm debugging approach
 - Clear articulation
 - Collaborative mindset
 - Ability to accept hints
-

Common Mistakes

Weak candidates:

- Start coding immediately
- Ignore constraints
- Ignore edge cases
- Cannot explain trade-offs
- Get stuck silently